

04_fresh_entries

June 29, 2026

1 04 — Fresh-entry detection (the flow engine)

Supersedes notebook 03 for live trading.

Notebook 03 looked at a *static snapshot* of who holds what. The live data proved the flaw: by the time 3 sharps visibly agree, the price has already moved (the Iran market ran $0.54 \rightarrow 0.90$ before consensus was visible). A snapshot is too late.

This notebook fixes that. Instead of “who holds what”, it asks “**who just entered, and can I still get in near their price?**” It reads each roster wallet’s recent BUY activity (last ENTRY_LOOKBACK_HOURS), finds markets where **several sharps entered the same side recently**, and keeps only those where the **current price is still close to what they paid**.

Run this **daily** (or more often). Requires `roster.json` from notebook 02.

```
[1]: import importlib, pmc, json, time
importlib.reload(pmc)
from pmc import (CFG, get_recent_buys, get_open_positions, portfolio_value,
                 position_size_eur)
import pandas as pd, numpy as np

roster = json.load(open("roster.json"))["wallets"]
skill_by_wallet = {r["wallet"]: r.get("skill", 0.5) for r in roster}
since_ts = int(time.time()) - CFG.ENTRY_LOOKBACK_HOURS * 3600
print(f"{len(roster)} roster wallets | looking back {CFG.ENTRY_LOOKBACK_HOURS}h")
↪
    f"(since unix {since_ts})")
```

27 roster wallets | looking back 72h (since unix 1782503387)

1.1 1. Pull recent BUYS + current price for every roster wallet

We join two trusted endpoints by token id (`asset`): - `/activity` → *when* they bought and at *what price* (the flow). - `/positions` → the *current* price and their conviction (only if they still hold it).

If a wallet already sold a position, we skip it — their exit means it is no longer a live entry signal.

```
[2]: rows = []
for r in roster:
    w = r["wallet"]
    buys = get_recent_buys(w, since_ts)
```



```

        return (df[val] * df[wt]).sum() / wsum if wsum else df[val].mean()

per_wallet = []
for (w, cid, outcome), g in buys_df.groupby(["wallet", "conditionId",
↪ "outcome"]):
    usdc = g["usdc"].sum()
    if usdc < CFG.MIN_TRADE_USDC:
        continue
    per_wallet.append({
        "wallet": w, "skill": g["skill"].iloc[0],
        "conditionId": cid, "outcome": outcome, "title": g["title"].iloc[0],
        "entry_price": round(wavg(g, "entry_price", "usdc"), 4),
        "usdc": round(usdc, 2),
        "last_ts": g["ts"].max(),
        "cur_price": round(g["cur_price"].iloc[0], 4),
        "conviction": g["conviction"].iloc[0],
    })
pw = pd.DataFrame(per_wallet)
print(f"{len(pw)} wallet-level fresh entries (>= ${CFG.MIN_TRADE_USDC:.0f})")

```

64 wallet-level fresh entries (>= \$500)

1.3 3. Group into fresh consensus per (market, outcome)

score = sum of backer skill (2 mid-skill sharps → ~0.8–1.0). dissent = roster sharps who recently bought the *opposite* side of the same market.

```

[4]: sig = []
for (cid, outcome), g in pw.groupby(["conditionId", "outcome"]):
    other = pw[(pw["conditionId"] == cid) & (pw["outcome"] != outcome)]
    sig.append({
        "conditionId": cid,
        "title": g["title"].iloc[0],
        "outcome": outcome,
        "backers": g["wallet"].nunique(),
        "dissenters": other["wallet"].nunique(),
        "score": round(g["skill"].sum(), 3),
        "median_entry": round(g["entry_price"].median(), 3),
        "cur_price": round(g["cur_price"].median(), 3),
        "total_usdc": round(g["usdc"].sum(), 0),
        "newest_hrs_ago": round((time.time() - g["last_ts"].max()) / 3600, 1),
    })
sig_df = pd.DataFrame(sig).sort_values(["backers", "score"], ascending=False)
print(f"{len(sig_df)} fresh (market, outcome) groups")
sig_df.head(20)

```

64 fresh (market, outcome) groups

[4]: conditionId \

```

2 0x178b29c0bb3a1c482f37f42e2420a1aa686b960ef000...
3 0x1f5b9dd89471579b5eef1bbee88a966e50a843d7f79e...
9 0x325d2f917428a0d39ac7563770ec519208666b8c9260...
10 0x348cd9adf4f6855f58bd9c6dbf9ff251c4142ef77233...
21 0x545e1bf6ec42e0e8a776a3c3e11ed225f8d3dfcc74f9...
26 0x79c850d5dd022d89aaf4f92105657d7a2e0518d4e4d4...
29 0x80026e4a0a2aad15549a47671a54aa95afc86325c604...
36 0x8e6c5f24da6e0a987c4b5cd73d3b341152ada808416a...
51 0xcacd424332319971356c757c025b45a4eb5643b919fb...
54 0xd73237bb27cdc455578e9a0788c358bd79609d394f43...
60 0xe99cc59f32b10d23acf196d1a0e8264ea30fca198428...
5 0x2235a83c801ecab8853b6dee650b149b103c178d2685...
16 0x487c0f8c01ebe72d23dbf69f3ef72908166cdcc6226d...
19 0x4e7ee488e38fe1944d61077b5f845798186cf0121be1...
23 0x6114f7db7f4c99210db8e54bc52a0aec1ef1fa7f15cd...
45 0xba9a25966e61386933732603c7eaa6a32290b5034f35...
1 0x08a4243912920a6f4e8d912f8b9e41c1e13340003ced...
13 0x394c280843cf929f6f7901feaada82a5d781ff049fd6...
17 0x4dea195cee059f398108c624b1db9505f8ec5dd99b4a...
20 0x5264e6b5489ac5c1c657c0001e54ac0f0ef262713ef8...

```

	title	outcome \
2	Will Claude Fable 5 be restored for US custome...	Yes
3	Will Manny Rutinel be the Democratic nominee f...	No
9	US x Iran diplomatic meeting by July 17, 2026?	No
10	Strait of Hormuz traffic returns to normal by ...	Yes
21	US x Iran diplomatic meeting by July 3, 2026?	No
26	US x Iran diplomatic meeting by July 10, 2026?	No
29	Will Melat Kiros be the Democratic nominee for...	No
36	Will Claude Fable 5 be restored for US custome...	Yes
51	Strait of Hormuz traffic returns to normal by ...	Yes
54	SCOTUS bars counting mail ballots after electi...	No
60	Will Colombia win the 2026 FIFA World Cup?	Yes
5	Spread: Seattle Mariners (-1.5)	Los Angeles Angels
16	Washington Nationals vs. Baltimore Orioles	Baltimore Orioles
19	Kansas City Royals vs. Chicago White Sox	Kansas City Royals
23	Houston Astros vs. Detroit Tigers	Detroit Tigers
45	San Diego Padres vs. Chicago Cubs	Chicago Cubs
1	Dota 2: Virtus.pro vs HULIGANI - Game 2 Winner	Virtus.pro
13	Dota 2: Nigma Galaxy vs Yellow Submarine - Gam...	Nigma Galaxy
17	Game Handicap: Na`Vi (-1.5) vs HULIGANI (+1.5)	Natus Vincere
20	Dota 2: Natus Vincere vs HULIGANI (BO3) - The ...	Natus Vincere

	backers	dissenters	score	median_entry	cur_price	total_usdc \
2	1	0	0.493	0.316	0.195	1181.0
3	1	0	0.493	0.055	0.065	823.0

9	1	0	0.493	0.253	0.345	532.0
10	1	0	0.493	0.049	0.009	1533.0
21	1	0	0.493	0.314	0.665	5861.0
26	1	0	0.493	0.268	0.510	553.0
29	1	0	0.493	0.217	0.225	5226.0
36	1	0	0.493	0.827	0.837	689.0
51	1	0	0.493	0.195	0.165	5589.0
54	1	0	0.493	0.783	0.994	10818.0
60	1	0	0.493	0.024	0.025	3546.0
5	1	0	0.362	0.530	0.535	3746.0
16	1	0	0.362	0.640	0.000	2100.0
19	1	0	0.362	0.450	0.000	12682.0
23	1	0	0.362	0.470	0.000	6892.0
45	1	0	0.362	0.590	0.595	1921.0
1	1	0	0.358	0.594	0.000	45569.0
13	1	0	0.358	0.681	0.000	14235.0
17	1	0	0.358	0.460	0.000	508.0
20	1	0	0.358	0.625	0.000	11874.0

	newest_hrs_ago
2	70.1
3	2.7
9	5.2
10	68.7
21	5.2
26	5.2
29	0.3
36	70.1
51	1.9
54	5.8
60	0.9
5	1.0
16	26.3
19	48.3
23	28.4
45	0.1
1	31.3
13	27.7
17	59.8
20	53.6

1.4 4. Gate + price guard (Blueprint \$4–\$5)

```
[5]: def gate(r):
      price_ok = (
          r["cur_price"] <= r["median_entry"] + CFG.SLIPPAGE_TOLERANCE and
          CFG.PRICE_FLOOR <= r["cur_price"] <= CFG.PRICE_CEILING
```

```

    )
    return (
        r["backers"]    >= CFG.MIN_FRESH_BACKERS and
        r["dissenters"] <= CFG.MAX_DISENTERS and
        r["score"]      >= CFG.FRESH_SCORE_CUTOFF and
        price_ok
    )

live = sig_df[sig_df.apply(gate, axis=1)].copy() if len(sig_df) else sig_df
print(f"{len(live)} fresh-entry signals pass the full gate")

```

0 fresh-entry signals pass the full gate

1.5 5. Suggested balanced sizing + save

Same 1/4-Kelly logic as the blueprint. A **proposal**, not an order — you execute.

```

[6]: def size_row(r):
    edge_margin = min(0.10, 0.04 * (r["score"] / CFG.FRESH_SCORE_CUTOFF))
    p_est = min(0.95, r["cur_price"] + edge_margin)
    eur = position_size_eur(p_est, r["cur_price"], r["score"], CFG.
    ↪ FRESH_SCORE_CUTOFF)
    return pd.Series({"p_est": round(p_est, 3), "suggested_eur": eur})

cols = [
    ↪ ["title", "outcome", "backers", "dissenters", "score", "median_entry", "cur_price",
        "newest_hrs_ago", "total_usdc", "p_est", "suggested_eur"]
if len(live):
    live = pd.concat([live, live.apply(size_row, axis=1)], axis=1)
    cap = CFG.MAX_TOTAL_DEPLOYED_PCT * CFG.BANKROLL
    live["cum"] = live["suggested_eur"].cumsum()
    live["suggested_eur"] = live.apply(lambda r: r["suggested_eur"] if r["cum"]
    ↪ <= cap else 0.0, axis=1)
    from datetime import datetime
    fname = f"fresh_signals_{datetime.now():%Y%m%d_%H%M}.csv"
    live[cols].to_csv(fname, index=False)
    print("saved", fname)
    display_cols = live[cols]
else:
    display_cols = "No fresh-entry signals right now. Re-run later - flow
    ↪ signals come and go."
display_cols

```

[6]: 'No fresh-entry signals right now. Re-run later - flow signals come and go.'

1.5.1 Why this is the right lens

The static engine (03) asked *who holds what* and almost always found the price had already moved. This engine asks *who just moved* and whether you can still follow — directly targeting the latency problem.

1.5.2 Honest caveat (still unproven)

Finding fresh overlapping entries is necessary but **not proof of edge**. Two sharps buying the same thing today does not guarantee it pays. The only way to know is the **backtest**: replay history, simulate “buy when N sharps entered within the window near current price”, and measure realized return vs. the market. Until then, treat these as *candidates to study and paper-trade*, not green lights.

1.5.3 Tuning knobs (in `pmc.py`)

ENTRY_LOOKBACK_HOURS (wider window → more signals, more stale), MIN_FRESH_BACKERS (2 is permissive; 3 is stricter), FRESH_SCORE_CUTOFF, MIN_TRADE_USDC.